

RetailMart Lakehouse

Project Design and Engineering Decisions

Implementation Summary

Project Design and Engineering Decisions

Rather than implementing individual notebooks independently, the project was designed following a modular and reusable architecture similar to production Data Engineering projects. The solution was organized into separate folders for Configuration, Utilities, Bronze, Silver, Gold, and Dashboard development. This modular design improved readability, reduced code duplication, and made future maintenance significantly easier.

A dedicated Configuration Notebook was created to centralize all project-level variables such as catalog names, schema names, storage paths, table names, and common Spark configurations. Instead of hardcoding these values in every notebook, all processing notebooks imported the configuration notebook. This approach ensures that if the project environment changes, only a single configuration file needs to be updated rather than modifying every notebook individually.

Similarly, a Utility Notebook was developed to store reusable helper functions and common operations that were required across multiple stages of the pipeline. By separating reusable logic from business transformations, the notebooks became cleaner, easier to understand, and more maintainable. This modular approach follows standard software engineering principles and reflects how enterprise-scale Data Engineering projects are typically structured.

Bronze Layer Implementation

The Bronze layer was designed to ingest raw CSV files exactly as received without modifying the original records. Instead of directly performing transformations during ingestion, the raw datasets were first stored as Delta Lake tables, preserving the source data for traceability and future reprocessing.

An additional engineering improvement was the use of a reusable ingestion framework driven by the centralized configuration notebook. Since all source locations, schemas, and table names were managed through configuration variables, the ingestion process became flexible and reusable rather than being tied to hardcoded paths.

Validation steps such as schema verification, record count validation, and table creation checks were also included after ingestion to ensure data integrity before moving to downstream processing.

Instead of discarding invalid records, dedicated quarantine tables were maintained to isolate records that failed validation. This ensured that valid data continued through the pipeline while problematic records remained available for investigation and future correction.

Audit columns were maintained to capture processing metadata, improving traceability and data lineage across the pipeline. Exception handling using try-catch blocks was implemented throughout the ingestion process to capture errors, simplify debugging, and improve pipeline reliability.

Silver Layer Implementation

The Silver layer focused on improving data quality and preparing standardized datasets for analytics. Duplicate records were removed, missing values were handled, inconsistent data formats were standardized, and business entities such as customers, products, sellers, and orders were

separated into dedicated analytical tables.

Beyond the expected cleaning operations, the Silver layer introduced Slowly Changing Dimension (SCD) Type 2) for product data. Instead of overwriting existing product information, historical versions were preserved using effective dates, expiry dates, and current record indicators. This enhancement enables historical analysis and reflects real-world enterprise data warehouse practices where maintaining change history is essential.

The Silver layer also leveraged Delta Lake's transactional capabilities to safely update records while maintaining data consistency during incremental processing.

Audit columns such as effective date, expiry date, processing timestamp, and current record indicators were maintained to support historical tracking and improve data governance.

Gold Layer Implementation

The Gold layer transformed cleaned operational data into business-ready analytical datasets. Instead of generating a single reporting table, multiple specialized Gold tables were created, each designed to answer a specific business question.

These included Fact Sales, Monthly Revenue, Customer 360, Customer Segmentation, Above Average Customers, Sales Funnel Analysis, Product Ranking, Customer Churn Risk, and Trending Products.

Several advanced SQL techniques were incorporated in this layer, including Common Table Expressions (CTEs), Aggregate Functions, CASE expressions, Joins, Date Functions, and Window Functions such as ROW_NUMBER(), RANK(), and DENSE_RANK(). These techniques enabled complex analytical calculations while maintaining efficient query execution.

Two significant enhancements were also implemented beyond the original project scope. A Customer Churn Risk model was developed using recency analysis to identify customers likely to discontinue purchasing, while a Trending Products model measured month-over-month growth to identify rapidly growing product categories rather than simply reporting highest-selling products.

Why Apache Spark?

Apache Spark was selected because it provides distributed data processing capabilities that are widely used in modern Data Engineering. Although the project dataset could have been processed using traditional SQL or Pandas, Spark was intentionally chosen to simulate production-scale ETL pipelines. Its DataFrame API, optimized execution engine, and seamless integration with Databricks made it well suited for implementing scalable transformations while following industry best practices.

Why Delta Lake?

Instead of storing processed data as ordinary CSV or Parquet files, the project utilized Delta Lake throughout the Medallion Architecture. Delta Lake provides several production-grade capabilities including ACID transactions, schema enforcement, schema evolution, version control, and reliable merge operations.

These capabilities became especially valuable while implementing SCD Type 2, where historical product records needed to be updated without compromising data consistency. Delta Lake ensured reliable updates while preserving historical versions, making the pipeline significantly more robust than a conventional file-based ETL solution.

Additional Enhancements Beyond Project Requirements

1. Production-Oriented Project Structure

Instead of keeping all logic inside individual notebooks, the project was organized into separate Configuration, Utilities, Bronze, Silver, Gold, and Dashboard modules. This modular architecture improves readability, code reuse, and long-term maintainability.

2. Centralized Configuration Management

A dedicated Configuration notebook was created to store catalog names, schema names, Delta table paths, and Spark configurations. Every notebook imports this configuration, eliminating hardcoded values and making the project portable across environments.

3. Reusable Utility Functions

Common helper functions and reusable operations were separated into a Utility notebook. This minimizes duplicate code and follows software engineering best practices for modular development.

4. Extensive Use of Delta Lake

Rather than storing transformed data as traditional files, every processing layer was implemented using Delta Lake tables. This enabled ACID transactions, schema enforcement, versioning, reliable MERGE operations, and efficient incremental processing throughout the pipeline.

5. Enhanced Slowly Changing Dimension (SCD Type 2) Implementation

The project requirements specifically required implementing SCD Type 2 for the Product dimension. However, instead of implementing a basic overwrite mechanism, a production-style historical tracking solution was developed using Delta Lake MERGE operations with effective dates, expiry dates, and active record indicators.

Additionally, the Silver layer was designed using incremental processing principles rather than recreating complete tables during every execution. This approach minimizes unnecessary processing while preserving historical information wherever applicable, making the pipeline significantly closer to enterprise Data Engineering practices.

6. Advanced Analytical Gold Layer

Beyond the required Gold tables, additional analytical datasets such as Customer Churn Risk and Trending Products were designed to answer real business questions that were not explicitly provided in the original requirements. These tables utilize advanced SQL features including Window Functions, CTEs, Ranking Functions, and analytical aggregations.

7. Interactive Executive Dashboard

Instead of creating only analytical tables, an Executive Dashboard was developed to visualize business KPIs, revenue trends, customer behavior, churn analysis, product performance, and geographical insights. This transforms the processed data into actionable business intelligence for decision-making.

8. Quarantine Tables

Instead of discarding invalid records, quarantine tables were maintained to isolate failed records for investigation and future correction while allowing valid data to continue through the pipeline.

9. Audit Columns

Audit columns were implemented to capture processing metadata, support data lineage, enable historical tracking, and improve governance across all processing layers.

10. Exception Handling using Try-Catch

Robust try-catch blocks were implemented throughout the notebooks to capture errors, simplify debugging, improve monitoring, and ensure reliable pipeline execution.